Student Guide: XAI Project — Evaluate Your AI Solution

> **What this is:** Your step-by-step guide to use the Explainable AI toolkit
> **GitHub Repo:** `github.com/RAI-Incubation-Lab/AXAI-Toolkit`

What You're Building

Your team has built (or chosen) an **AI solution** — maybe it's a classifier you trained, a ChatGPT API integration, a Claude-powered tool, or a no-code AI platform. This project asks you to **evaluate its explainability**: can you show why it made a specific decision, whether it's fair, and what would need to change to get a different result?

You produce an **XAI Report Card** — a document that scores and explains how interpretable, fair, and auditable your AI solution is. Think of it like a nutrition label, but for AI.

Key idea: You don't need to have trained the model yourself. You need to understand and explain whatever AI is driving your solution.

Step 1: Get Started (Day 1)

Open the Master Template

Download `templates/AXAI_Master_Template.ipynb` from the repo. Open it in whatever environment you prefer:

- **Google Colab:** Upload the `.ipynb` to your Drive, open it
- **Local Jupyter:** `jupyter notebook AXAI_Master_Template.ipynb`
- **Baidu AI Studio:** Upload and open

Run Cell 1 (Master Import Cell) first — it installs all required libraries (~2 minutes).

Clone the Demo

See what a finished project looks like: `Case_Studies/demo-titanic/` in the repo.

This is a complete example using the Titanic dataset. Run it, explore it, understand the structure. This is what your submission should look like.

Step 2: Describe Your AI Solution (Days 1-2)

Fill in Cell 2 of the Master Template

```
solution_name = "Loan Risk Classifier"
solution_type = "classification"                  # classification | regression | text_generation
tech_stack = "OpenAI GPT-4 API + custom prompts"  # What AI tools did you use?
dataset_description = "Loan application dataset"  # What data does your solution process?
target_variable = "loan_approved"                  # What does your solution predict or decide?
```

Document Your Tech Choices

In your Model Card and README, answer:

- **What AI tools did you use?** (APIs, libraries, no-code platforms, open-source models)
- **Why did you choose them?** (accuracy, cost, speed, availability, ease of use)
- **What alternatives did you consider?** (Why GPT-4 over Claude? Why RandomForest over XGBoost?)
- **Who are the end users?** Who benefits? Who could be harmed by wrong answers?
- **Project context:** How does your AI solution's explainability approach account for regional framework baselines — such as GBA cross-border data policies, or regional healthcare governance and privacy structures?

This isn't about whether your tool is the "best." It's about whether you can **explain and justify** your choices in the context where the solution would actually be deployed.

Step 3: Connect Your Solution to the Template (Days 2-4)

Cell 3: Load Your Test Data

```
user_dataset = pd.read_csv("your_data.csv")    # ← change this
target_column = "your_target_column"            # ← change this
```

Load whatever data your solution processes. It can be anything tabular — customer records, student submissions, patient data.

Cell 4: Get Predictions from Your Solution

This is the key cell. The template needs to collect predictions from your solution so it can explain them. Choose the path that matches your setup:

**PATH A: You trained a model

```
predictions = user_model.predict(X)
probabilities = user_model.predict_proba(X)
```

Just plug in your model. Done.

PATH B: You use an API (OpenAI, Claude, Gemini, etc.)

```
def call_ai_api(row):
    """Send each row to your AI API, return the prediction."""
    prompt = f"Based on these inputs, predict the outcome: {row.to_dict()}"
    response = openai.chat.completions.create(
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

predictions = X.apply(call_ai_api, axis=1)
```

Write a function that sends each data row to your API and returns the result. SHAP and LIME

will automatically fall back to `KernelExplainer` (slower but universal — it works by perturbing inputs and observing outputs).

PATH C: You use a no-code tool

```
# Export predictions from your tool as CSV, then load:
predictions = pd.read_csv("predictions.csv")["predicted_column"]
```

If your tool doesn't have an API, run your data through it separately, export the predictions, and load them here. Explainability analysis still works — it just won't have access to prediction probabilities.

Step 4: Review Your Explainability Results (Days 5-8)

After running Cells 5-9, you'll get:

SHAP Output (Cell 5)

- **Summary plot** — which input features drive predictions overall
- **Waterfall plot** — why one specific decision was made
- Works with any solution type (auto-falls back to KernelExplainer for APIs)

LIME Output (Cell 6)

- **Local explanation** — what features drove one individual decision
- Shows the top 5 features and how they pushed the prediction

Fairness Audit (Cell 7)

- **Demographic Parity** — does the solution treat groups equally?
- **Equalized Odds** — are error rates similar across groups?
- You need to identify at least one sensitive attribute (gender, age, region, etc.)

Counterfactual Examples (Cell 8)

- "Change feature X by Y% to flip the prediction"
- Shows what minimum changes would produce a different outcome

Model Card (Cell 9)

- Auto-generated markdown document with all metrics, findings, and limitations
- If the modelcard package isn't installed, a fallback template still produces a usable report

Your job now: Read through the results. What surprised you? Are there fairness concerns? Do the explanations make business sense? Do SHAP/LIME actually make sense for your solution type? Document your findings in the Model Card.

Step 5: Build Your Dashboard (Days 9-14)

Build a Streamlit or Gradio app so a non-technical person can interact with your solution and see explanations.

Your dashboard must show:

1. **Input form** — user enters data, gets a prediction
2. **SHAP explanation** — why that prediction (waterfall or force plot)
3. **Counterfactual demo** — "what would need to change?"
4. **Model Card viewer** — display the full report

Minimal Streamlit App (`app.py`)

```
import streamlit as st
import shap

st.title("XAI Report Card — [Your Project]")

# Input form
st.header("Make a Prediction")
# ... build your input form ...

# Prediction + explanation
if st.button("Predict"):
    prediction = get_prediction(input_values)
    st.write(f"Prediction: {prediction}")

    # SHAP explanation
    st.header("Why this prediction?")
    shap_values = explainer.shap_values([input_values])
    st.pyplot(shap.force_plot(...))

    # Counterfactual
    st.header("What would change the result?")
    # ... show counterfactual ...
```

Test before deploying

```
streamlit run app.py
```

Step 6: Complete Your Model Card (Days 12-14)

Review the auto-generated Model Card (Cell 9 output). Complete all sections:

- [ ] All metrics are filled in (no empty fields)
- [ ] Tech choices are justified (why this API/model over alternatives)

- [ ] Top features from SHAP are listed and explained
- [ ] Counterfactual examples are meaningful and documented
- [ ] Fairness results are discussed (even if bias was found — that's OK)
- [ ] Known limitations are honestly listed
- [ ] Stakeholder notes are filled in

Step 7: Submit via GitHub PR (Days 15-16)

Your submission structure:

```
Case_Studies/[your-team-name]/
├── your-project.ipynb          # Your completed Colab notebook
├── MODEL_CARD.md               # Your completed Model Card
├── app.py                      # Your Streamlit dashboard
├── README.md                   # How to reproduce your solution
├── demo.mp4                    # 2-minute screen recording
└── requirements.txt            # pip requirements
```

How to Submit

1. Fork `RAI-Incubation-Lab/AXAI-Toolkit`
2. Create a branch: `git checkout -b team-[your-name]`
3. Add your folder: `git add Case_Studies/[your-team-name]/`
4. Commit: `git commit -m "Add [your-team-name] XAI project"`
5. Push: `git push origin team-[your-name]`
6. Create a Pull Request on GitHub

The PR Template

When you create a PR, a template will appear. Fill in every checkbox:

- [ ] Notebook runs end-to-end without errors
- [ ] SHAP plots present and interpreted
- [ ] LIME explanation demonstrated

- [ ] Counterfactual examples shown
- [ ] Fairness audit completed
- [ ] Dashboard functional
- [ ] Model Card fully populated
- [ ] README and demo included

Grading (100 points total)

Criteria	Points
Solution documentation & tech choice justification	15
SHAP analysis completeness	15
LIME explanation quality	10
Counterfactual examples	10
Fairness audit thoroughness	15
Dashboard functionality	15
Model Card completeness	10
Code quality & documentation	10
Total	**100**

Full rubric: See `templates/grading_rubric.md` in the repo.

Pre-Submission Checklist

Before you submit:

- [ ] All AI tools/APIs documented and justified
- [ ] End users and potential harm identified

- [ ] Regulatory frameworks considered (GDPR, HKMA, GBA cross-border data policies, etc.)
- [ ] SHAP global + local analysis present
- [ ] LIME local explanation demonstrated
- [ ] At least 2 counterfactual examples shown
- [ ] Sensitive attributes identified and tested
- [ ] Fairness metrics computed and discussed
- [ ] Model Card has no empty fields
- [ ] Dashboard runs without errors
- [ ] README explains how to reproduce
- [ ] Demo video included
- [ ] Notebook runs top-to-bottom without errors
- [ ] All TODOs replaced
- [ ] Citation block included

Full checklist: See `templates/checklist_template.md` in the repo.

FAQ

"I didn't train a model — I just used ChatGPT API. Can I still do this?"
Yes. Use PATH B in Cell 4 to call the API. SHAP/LIME will use KernelExplainer (it perturbs inputs and watches outputs — works with any black box). The explanation will be slower but still meaningful.

"My no-code tool doesn't have an API."
Use PATH C. Export predictions as CSV and load them. You won't get prediction probabilities, but you'll still get feature importance and counterfactuals.

"SHAP/LIME don't make sense for my solution type — what do I do?"
That's a valid finding. Document why they don't apply and what explainability method would be more appropriate. This is part of the grading criteria (tech choice justification).

"My fairness audit shows bias — is that bad?"
No. Finding bias is a valid result. Document it, explain why it exists, propose mitigations. That's exactly what the project is testing.

"My Streamlit app works locally but not when I push it."
Check `requirements.txt` — make sure all dependencies are listed.

"Can I use any dataset?"
Yes. Any tabular dataset with a clear target variable works.

"What if my API has rate limits?"
Use a smaller sample (`X.iloc[:50]`) for explainability analysis. You don't need all 1000 rows to get meaningful SHAP/LIME results.

Resources

Resource	What
`templates/AXAI_Master_Template.ipynb`	Your starting notebook (open in Colab/Jupyter/ your env)
`Case_Studies/demo-titanic/`	Complete worked example
`templates/model_card_template.md`	Model Card structure
`templates/checklist_template.md`	Pre-submission checklist
`templates/grading_rubric.md`	How you'll be graded
`.github/PULL_REQUEST_TEMPLATE.md`	What your PR must include